Pallegama Stores — Customer Mobile App API

Everything the customer-facing mobile app needs. All endpoints live under the `/api` prefix of the backend (Laravel + Sanctum).

Base URL: `https://<server-host>/api` — ask the store admin for the host.

Common to every request:

Header	Value
`Accept`	`application/json`
`Content-Type`	`application/json` (on POST / PUT)
`Authorization`	`Bearer <token>` (only on protected endpoints)

---

1. Authentication

The app authenticates as a **customer**, not as staff. Login returns a Sanctum bearer token carrying the `customer` ability. That token works **only** on the customer endpoints in this document; it is rejected everywhere else in the ERP.

Store the token securely on the device (Keychain / EncryptedSharedPreferences) and send it as `Authorization: Bearer <token>`.

POST `/customer/register`

Open (no token). Creates the customer account.

```
{
  "name": "Nimal Perera",
  "mobile": "0771234567",
  "email": "nimal@example.com",
  "date_of_birth": "1990-04-17",
  "address": "12, Main Street, Pallegama",
  "password": "secret123",
  "password_confirmation": "secret123"
}
```

Field	Rules
`name`	required, max 255
`mobile`	required, max 30, **must be unique**
`email`	optional, valid email, must be unique
`date_of_birth`	optional, `YYYY-MM-DD` — drives birthday offers
`address`	optional, max 255
`password`	required, min 8, must match `password_confirmation`

201 Created

```
{
  "customer": { "debtor_no": 42, "name": "Nimal Perera", "mobile": "0771234567", "debtor_ref":
"CUST-A7B2C9", "...": "..." },
  "token": "17|AbCdEf..."
}
```

A new account is a **normal customer** — it does **not** get a loyalty card. Right after sign-up, `GET /customer/me/loyalty` returns **404** and `loyalty_card` in `GET /customer/me` is `null`.

The customer becomes a **loyalty customer** only when store staff upgrade them (back office: Customers → Normal Customers → **Make Loyalty**). They then get a card on the lowest tier (Silver, 0 points). The app has no endpoint to do this itself — the customer visits the store.

	Normal customer	Loyalty customer
`GET /customer/me` → `loyalty_card`	`null`	card object with `tier`
`GET /customer/me/loyalty`	**404**	card, points, tier, QR
`GET /customer/me/offers` → `for_you`	always `[]`	tier, personal and birthday offers
`GET /customer/me/offers` → `other_offers`	store-wide offers	store-wide offers
Promotions, purchases, receipts, notifications	yes	yes

Re-fetch `GET /customer/me` when the app opens: if `loyalty_card` is no longer `null`, the staff have upgraded the customer — switch the app to the loyalty view.

422 — validation errors, or `{"message": "Store is not fully set up yet ..."}` if the store company setup is incomplete. Show the `message` as-is.

POST `/customer/login`

Open. `login` accepts **either the mobile number or the email**.

```
{ "login": "0771234567", "password": "secret123" }
```

200 — same shape as register (`customer` + `token`). **422** — `{"errors": {"login": ["Invalid credentials."]}}` **403** — `{"message": "This account is inactive."}`

POST `/customer/logout`

Protected. Revokes **only the current device token**, so logging out on the phone does not sign the customer out elsewhere.

200 `{"message": "Logged out successfully"}`

POST `/customer/forgot-password`

Open. Requires an email on the account (mobile-only accounts cannot reset this way yet).

```
{ "email": "nimal@example.com" }
```

200 / **422** — both return `{"message": "..."}` ; display it directly.

POST `/customer/reset-password`

Open. `token` is the one from the reset email.

```
{
  "token": "<from email>",
  "email": "nimal@example.com",
  "password": "newsecret123",
  "password_confirmation": "newsecret123"
}
```

---

2. Protected endpoints (`Authorization: Bearer <token>` required)

A missing or non-customer token gives **401** `{"message": "Unauthenticated."}` . A token without the `customer` ability gives **403** `{"message": "Forbidden."}` . Treat **401 as "log the user out and show the login screen."**

GET `/customer/me`

The profile of the logged-in customer, with the loyalty card and tier eager-loaded.

```
{
  "debtor_no": 42,
  "name": "Nimal Perera",
  "mobile": "0771234567",
  "email": "nimal@example.com",
  "address": "12, Main Street, Pallegama",
  "date_of_birth": "1990-04-17",
  "debtor_ref": "CUST-A7B2C9",
  "last_purchase_date": "2026-10-28",
  "loyalty_card": {
    "id": 5,
    "debtor_no": 42,
    "card_no": "LC00042",
    "issue_date": "2025-06-01T00:00:00.000000Z",
    "loyalty_tier_id": 4,
    "points_balance": "1250.00",
    "status": "active",
    "created_at": "2025-06-01T10:10:17.000000Z",
    "updated_at": "2026-10-28T09:15:02.000000Z",
    "tier": {
      "id": 4,
      "tier_name": "Gold Member",
      "min_spend_threshold": "5000.00",
      "points_earn_rate": "0.0200",
      "redemption_rate": "1.0000",
      "benefits_description": "Double points on all purchases! Free home delivery for orders over
5,000.",
      "inactive": false,
      "created_at": "2026-09-04T07:49:00.000000Z",
      "updated_at": "2026-09-04T07:49:00.000000Z"
    }
  }
}
```


`loyalty_card` is `null` for a **normal customer** (not upgraded by the store yet — see the register notes above). Use it to decide which view to show: `null` → normal view; a card → loyalty view with points and tier. The tier object is the same as in `GET /customer/me/loyalty` below — show `tier.tier_name`.

PUT `/customer/me`

Send only the fields being changed. All four are optional.

```
{ "name": "Nimal Perera", "email": "new@example.com", "address": "...", "date_of_birth": "1990-04-
17" }
```

200 — the updated customer object. **422** if the email is taken.

POST `/customer/me/change-password`

```
{ "current_password": "secret123", "password": "newsecret123", "password_confirmation":
"newsecret123" }
```

200 `{"message": "Password updated."}` **422** `{"errors": {"current_password": ["Current password is incorrect."]}}`

GET `/customer/me/loyalty`

```
{
  "card_no": "LC00042",
  "qr_payload": "LC00042",
  "points_balance": "1250.00",
  "tier": {
    "id": 4,
    "tier_name": "Gold Member",
    "min_spend_threshold": "5000.00",
    "points_earn_rate": "0.0200",
    "redemption_rate": "1.0000",
    "benefits_description": "Double points on all purchases! Free home delivery for orders over
5,000.",
    "inactive": false,
    "created_at": "2026-09-04T07:49:00.000000Z",
    "updated_at": "2026-09-04T07:49:00.000000Z"
  },
  "status": "active",
  "issue_date": "2025-06-01T00:00:00.000000Z"
}
```

Tier fields:

Field	Meaning
`tier_name`	Name to show, e.g. "Silver Member", "Gold Member", "Platinum Member".
`min_spend_threshold`	Total spend needed to reach this tier.
`points_earn_rate`	Points earned per 1 rupee spent (0.02 = 2 points per 100).
`redemption_rate`	Rupee value of 1 point.
`benefits_description`	Benefits text to show under the tier.

Numbers come as strings — parse them before doing maths. `tier` can be `null` if the card has no tier set.

A new loyalty customer starts at Silver. The tier moves up automatically: after each sale the customer is moved to the highest tier their total spend **since becoming a loyalty customer** (`issue_date`) has reached

— spending as a normal customer does not count. Never moved down. Re-fetch this endpoint when the app opens or the screen is refreshed to show the current tier.

Render `qr_payload` as a QR code — the cashier scans it at the POS to attach the sale to this customer and award points.

404 `{"message": "No loyalty card found for this account."}` — this is a **normal customer** (the normal state right after sign-up). Show a "Visit the store to become a loyalty customer" message, not an error.

GET `/customer/me/loyalty/history`

Array, newest first, max 200 rows.

```
[
  {
    "id": 310,
    "debtor_no": 42,
    "debtor_trans_no": 1051,
    "debtor_trans_type": 10,
    "points_earned": "35.00",
    "points_redeemed": "0.00",
    "balance_after": "1250.00",
    "transaction_date": "2026-10-28"
  }
]
```

Each row is either an earn (`points_earned` > 0) or a redeem (`points_redeemed` > 0). `balance_after` is the running balance.

GET `/customer/me/offers`

Two buckets:

```
{
  "for_you": [ { "id": 7, "offer_name": "Silver 10% off", "offer_type": "tier", "discount_type":
"percentage", "discount_value": 10, "valid_from": "2026-11-01", "valid_to": "2026-11-30",
"image_path": "offers/x.jpg" } ],
  "other_offers": [ { "id": 9, "offer_name": "Rice 5% off", "offer_type": "product", "...": "..."
} ]
}
```

- `for_you` — offers matched to **this** customer: their loyalty `tier`, `customer`-specific offers, and a `birthday` offer on their birthday. **Loyalty customers only** — for a normal customer it is always `[]`.
- `other_offers` — all active `product` / `category` offers (store-wide).

`offer_type` is one of `product`, `category`, `customer`, `tier`, `birthday`. `discount_type` is `percentage` or a fixed amount.

GET `/customer/me/purchases`

The purchase history of the logged-in customer.

Query params (all optional): `from_date=YYYY-MM-DD`, `to_date=YYYY-MM-DD`, `limit` (default 100, capped at 500).

Returns an array of `debtor_trans` rows, newest first. Key fields:

```
[
  {
    "trans_no": 1051,
    "trans_type": 10,
    "debtor_no": 42,
    "tran_date": "2026-10-28",
    "reference": "INV-1051",
    "ov_amount": 4500.00,
    "ov_discount": 100.00,
    "ov_gst": 0.00,
    "alloc": 4500.00,
    "customer_name": "Nimal Perera"
  }
]
```

Receipt total = `ov_amount + ov_gst - ov_discount`. `trans_type` 10 is an invoice, 11 a credit note, 12 a payment.

GET `/customer/me/purchases/{transNo}`

One receipt, with lines. The backend verifies the receipt belongs to the logged-in customer, so a wrong or foreign `transNo` gives **404**, never someone else's data.

```
{
  "header": { "trans_no": 1051, "trans_type": 10, "tran_date": "2026-10-28", "ov_amount": 4500.00,
"...": "..." },
  "lines": [
    { "id": 8801, "debtor_trans_no": 1051, "stock_id": "1000", "description": "සුදු සීනි 1kg",
"quantity": 2, "unit_price": 250.00, "discount_percent": 0 }
  ]
}
```

404 `{"message": "Receipt not found."}`

GET `/customer/me/notifications`

Array, newest first, max 100.

```
[
  {
    "id": 55,
    "debtor_no": 42,
    "type": "offer",
    "title": "Happy Birthday!",
    "body": "Enjoy 15% off today.",
    "data": { "offer_id": 7 },
    "read_at": null,
    "created_at": "2026-11-03T04:10:00.000000Z"
  }
]
```

`read_at: null` means unread — use that for the badge count. `data` is a free-form JSON object; read its keys defensively, it varies by `type`.

POST `/customer/me/notifications/{id}/read`

Marks one notification read. **200** returns the updated notification. **404** if the id does not belong to this customer.

GET `/customer/me/offers-active`

Same payload as the public `/mobile/offers` below. Convenience route if you are already sending the token.

---

3. Public endpoints (no token)

Useful for the pre-login home screen.

GET `/mobile/offers`

```
{
  "data": [
    { "id": 9, "offer_name": "Rice 5% off", "offer_type": "product", "discount_type":
"percentage", "discount_value": 5, "valid_from": "2026-11-01", "valid_to": "2026-11-30",
"image_path": "offers/rice.jpg" }
  ]
}
```

Only offers that are `active` **and** valid today. Note the payload is wrapped in `data`.

GET `/mobile/promotions`

Banners and promotions curated by the store for the app, already sorted by `sort_order`.

Optional query param: `type=<promo_type>` to filter.

```
{
  "data": [
    {
      "id": 3,
      "title": "November Mega Sale",
      "description": "Up to 20% off groceries.",
      "promo_type": "banner",
      "discount_type": "percentage",
      "discount_value": 20.0,
      "valid_from": "2026-11-01",
      "valid_to": "2026-11-30",
      "image_path": "promotions/nov.jpg",
      "image_url": "https://<host>/api/storage-file/promotions/nov.jpg",
      "sort_order": 1
    }
  ]
}
```

Use **`image_url`** — it is a ready-to-load absolute URL. Only currently-active promotions are returned.

GET `/mobile/promotions/{id}`

200 `{"data": { ...same shape... }}` **404** `{"message": "Promotion not found or expired"}`

GET `/storage-file/{path}`

Serves an uploaded image. `path` is the `image_path` value from an offer or a promotion. For promotions prefer the pre-built `image_url`; for **offers** build it yourself:

```
https://<host>/api/storage-file/{offer.image_path}
```

No token needed, so it drops straight into any image widget.

---

4. Errors

Status	Meaning	App should
200 / 201	Success	—
401	Missing or invalid token	Clear the token, go to login
403	Wrong token type, or inactive account	Show `message`, log out
404	Not found, or not a record of this customer	Show empty state
422	Validation failed	Show field errors
500	Server error	Generic retry message

422 always looks like this — map `errors` onto your form fields:

```
{
  "message": "The mobile has already been taken.",
  "errors": { "mobile": ["The mobile has already been taken."] }
}
```

---

5. Notes and current limits

- Money comes back as a number or as a decimal **string** (e.g. `"1250.00"`). Parse it; do not assume one type. Currency is LKR.
- Dates are `YYYY-MM-DD`; timestamps are UTC ISO-8601.
- Item descriptions are often **Sinhala** — make sure the app fonts render Sinhala correctly.
- There is **no push notification endpoint yet**. `/customer/me/notifications` is poll-based; poll it on app resume. Push plus device-token registration is planned, not built.
- There is no product catalogue or online-ordering API yet. The app is account + loyalty + offers + purchase history.
- `debtor_no` is the permanent customer id — use it as the local user key.
- **Tokens expire after 7 days** (server setting, may change). Nothing warns the app beforehand: the next call simply returns **401**. Handle 401 globally — clear the stored token and send the user to the login screen. Do not treat it as a crash, and do not keep retrying the same token.
- There is **no refresh-token endpoint**. Re-login is the only way to get a new token, so consider keeping the user signed in with a saved credential prompt (biometric / remembered password) rather than making them retype it weekly.
- `/customer/forgot-password` only works once the server's SMTP mail is configured. If it returns a 422 message about mail, it's a server setup issue, not an app bug.

6. Quick smoke test

```
HOST=https://your-host

# Public — should return JSON with no token
curl "$HOST/api/mobile/offers"

# Register
curl -X POST "$HOST/api/customer/register" \
  -H "Accept: application/json" -H "Content-Type: application/json" \
  -d '{"name":"Test
User","mobile":"0770000001","password":"secret123","password_confirmation":"secret123"}'

# Login, then call a protected route
TOKEN=$(curl -s -X POST "$HOST/api/customer/login" \
  -H "Accept: application/json" -H "Content-Type: application/json" \
  -d '{"login":"0770000001","password":"secret123"}' | jq -r .token)

curl "$HOST/api/customer/me" -H "Accept: application/json" -H "Authorization: Bearer $TOKEN"
```